

Hello, Ich bin

João Bastos

Frontend Entwickler

Python React Javascript PHP

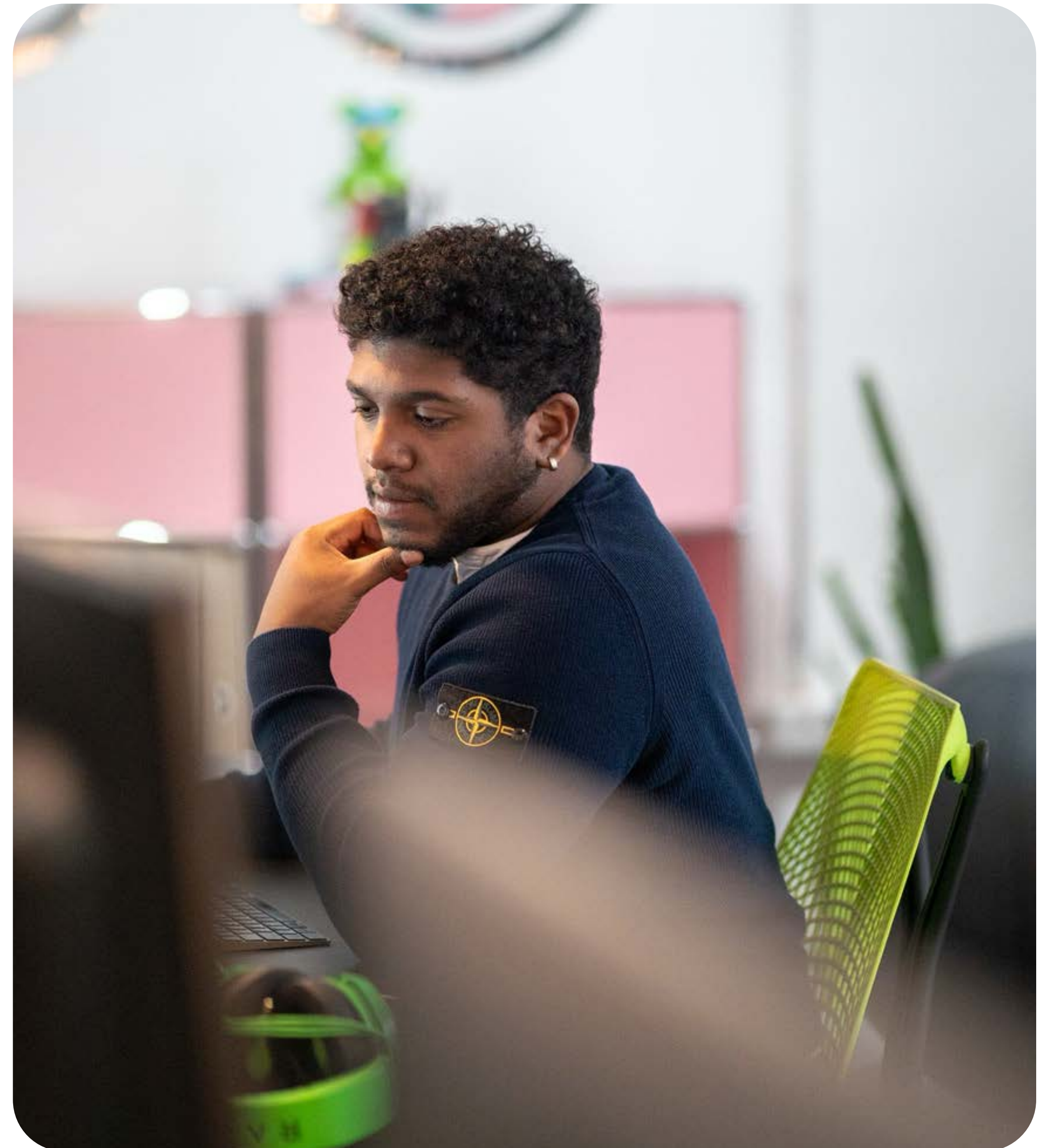
Ich bin ein brasilianischer Frontend-Entwickler und lebe seit 2017 in Düsseldorf, wo ich auch meine Ausbildung gemacht habe.

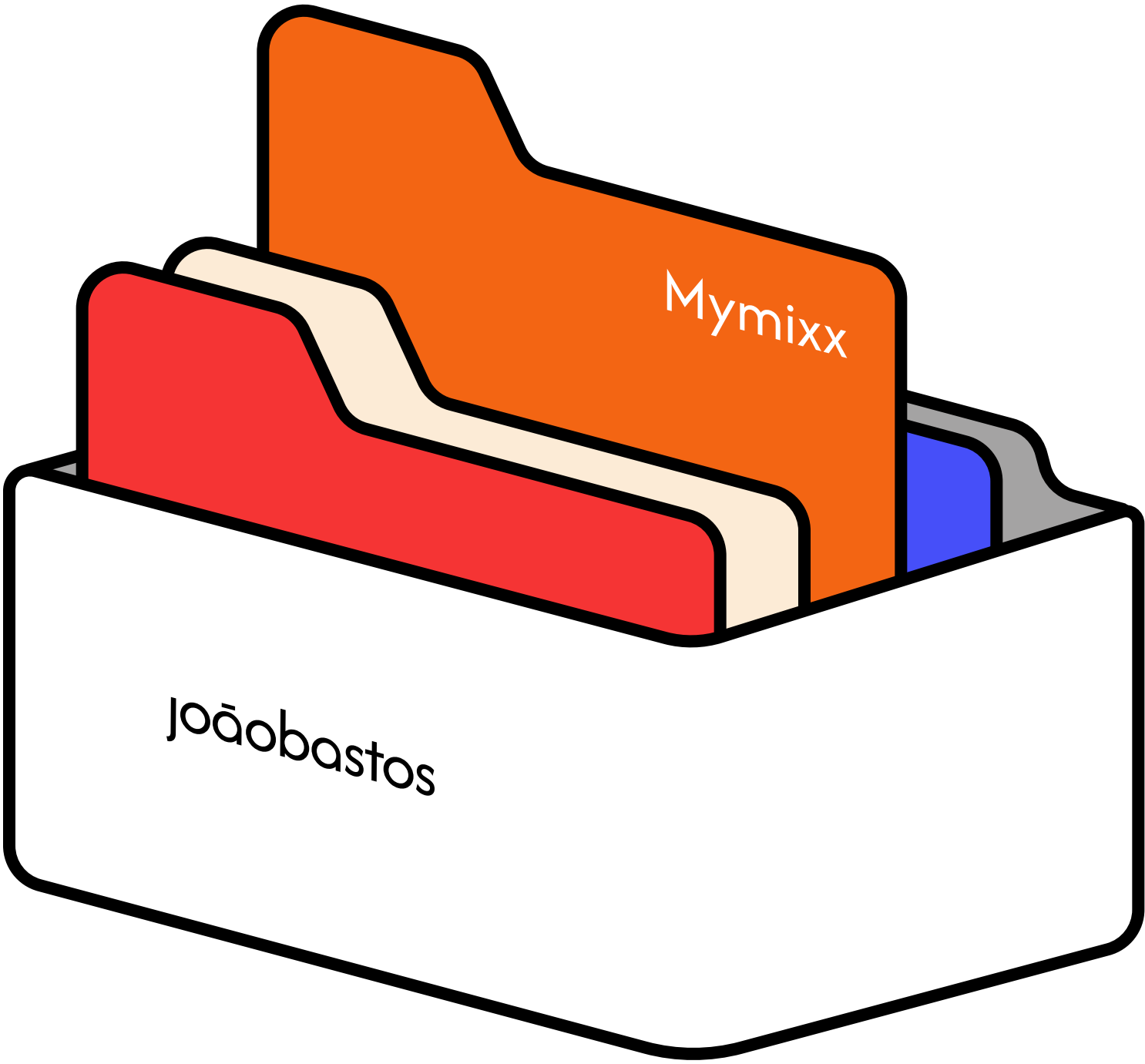
Ich bin ein eher ruhiger, aber kommunikativer Typ, der mit Leidenschaft programmiert, gerne Schach spielt und sich über Popkultur austauscht.

email: joaobastos@outlook.de



Schaue dir den Portfolio in meiner Website

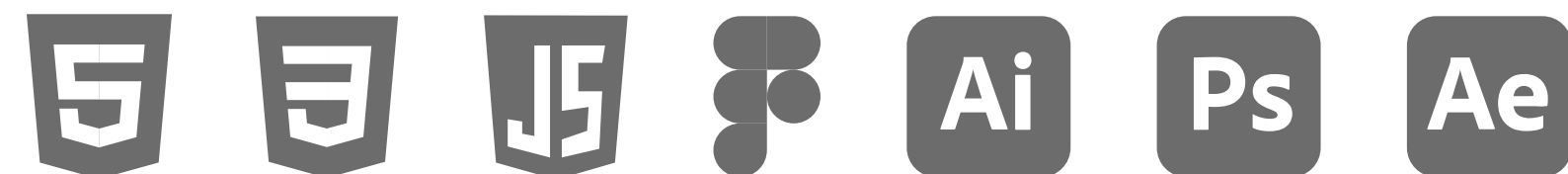






Mymixx ist ein Müsli-Store, in dem sich jeder sein persönliches Müsli zusammenstellen kann. Von der Basis bis zu den Toppings.

Zu meinen Aufgaben gehörte die Erweiterung der visuellen Identität, die digitale Umsetzung des Brandings sowie die Gestaltung und Programmierung des Bestellsystems.



Design

Bunt. Vielfältig. Möglich

Für das Design habe ich eine bunte und vielfältige Bildsprache entwickelt, die die zahlreichen Mix-Möglichkeiten widerspiegelt. Die lebendige Farbwelt betont die Pluralität der Zutaten und vermittelt visuell dieselbe Vielfalt und Freiheit, die auch die Marke selbst verkörpert.

Orange

C: 49 R: 243
M: 61 G: 101
Y: 60 B: 19
K: 66
#F36513

Grenn

C: 49 R: 154
M: 61 G: 191
Y: 60 B: 17
K: 66
#9ABF11

Yellow

C: 49 R: 242
M: 61 G: 187
Y: 60 B: 22
K: 66
#F2B816

Blue

C: 49 R: 34
M: 61 G: 221
Y: 60 B: 242
K: 66
#22DDF2

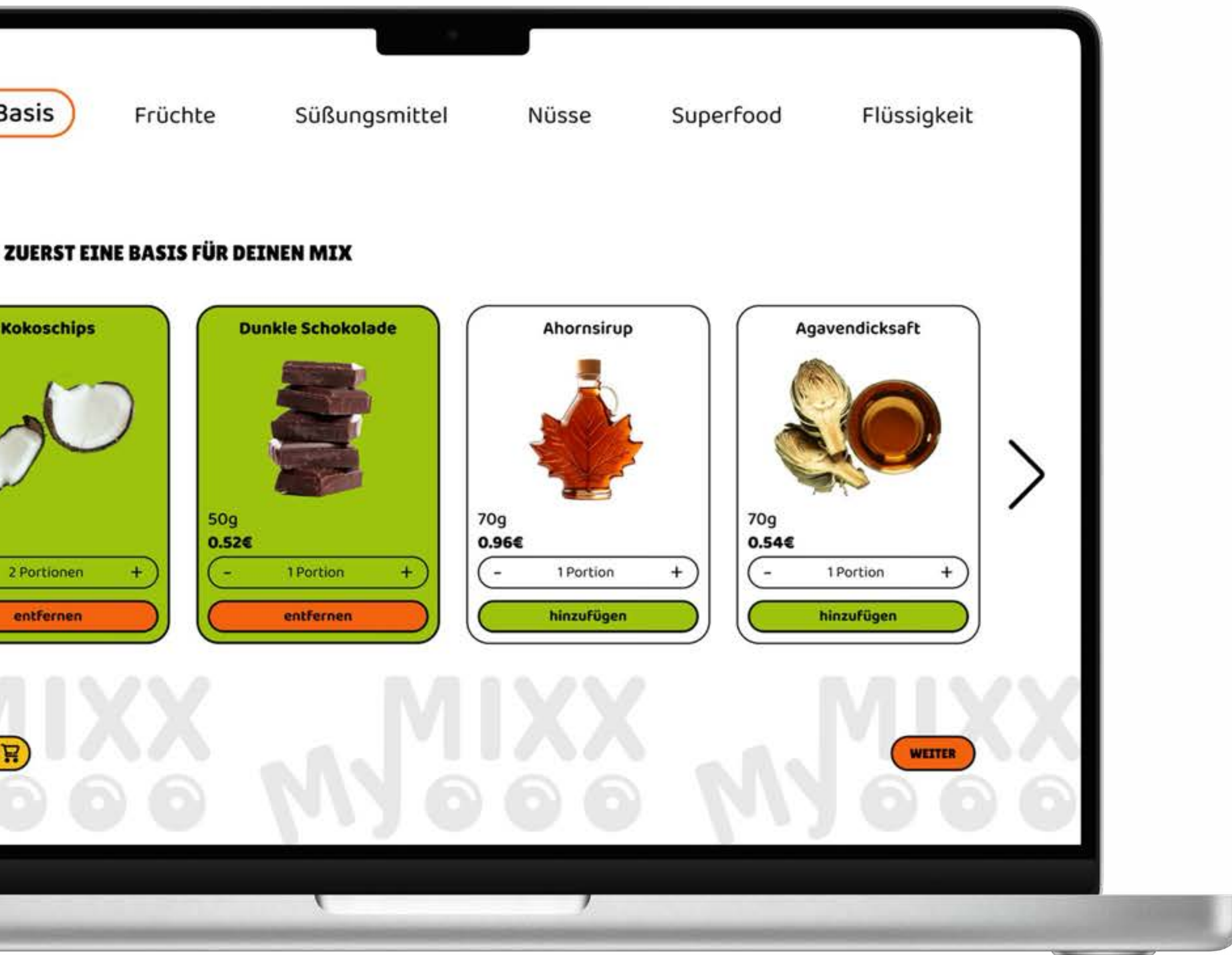
Black

C: 0 R: 0
M: 11 G: 0
Y: 40 B: 0
K: 0
#000000

Aa

Google Font
**Baloo
Thambi**

Aa Bb Cc Dd Ed FF Gg Hh Ii
Jj Kk Ll Mm Nn Oo Pp Qq Rr
Ss Tt Uu Vv Ww Xx Yy Zz
1 2 3 4 5 6 7 8 9 0



Code

Architektur von Datenbank

Der Schwerpunkt lag darauf, die Produktdaten klar zu strukturieren und die UI-Elemente dynamisch zu generieren.

Um die Produkte zu organisieren, habe ich ein JavaScript Datei erstellt, wo ich die Array von Objekten mit allen relevanten Informationen enthalten(Name, Preis, Menge und Bild) enthält. Dadurch wird sowohl die Pflege als auch das Rendering im Frontend vereinfacht.

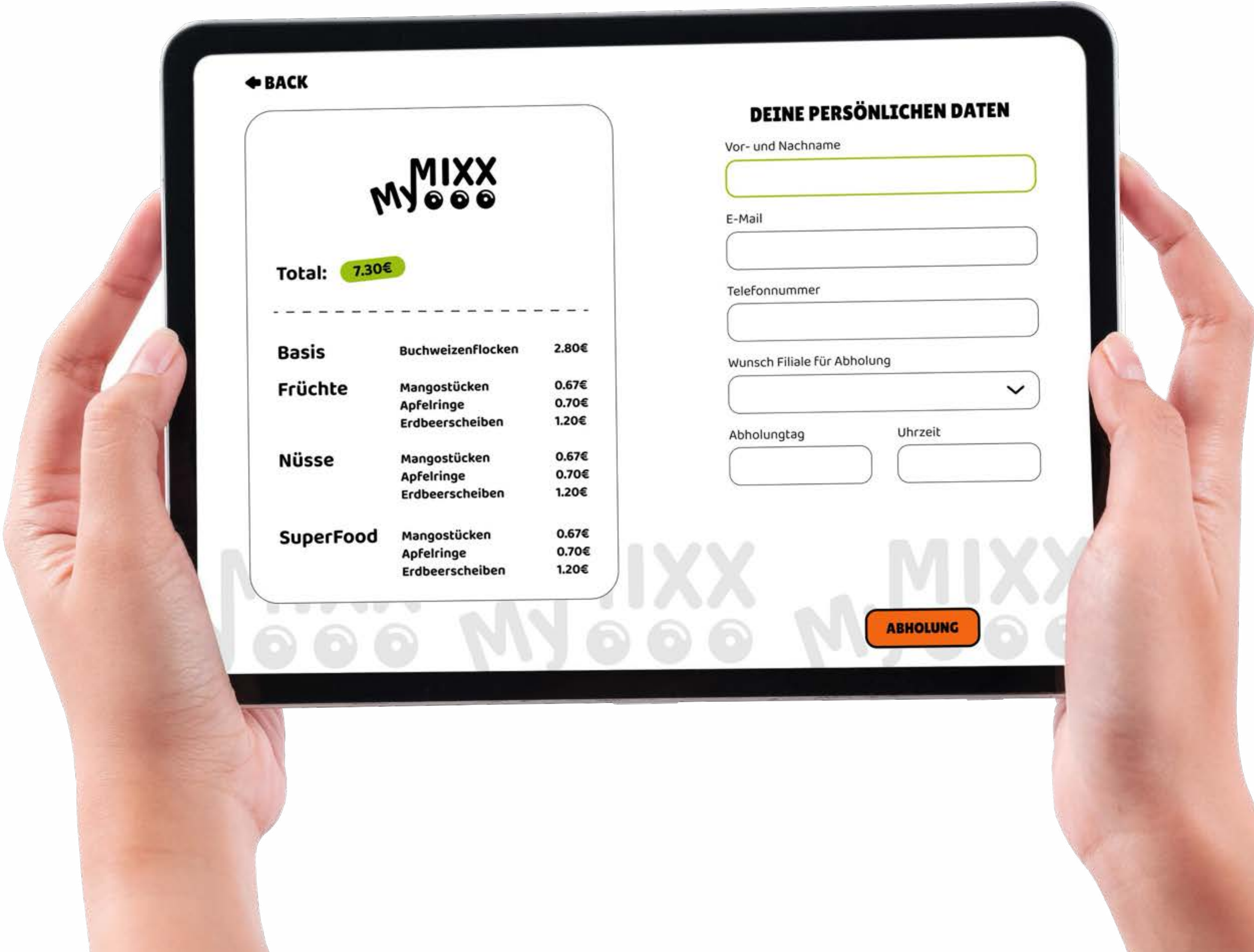
Die Struktur in dem Jason Datei wird in Objects kategorisiert
Anbei ist ein Beispiel, wie die Früchtensorte sortiert wird:

```
1  const arrayFruechte = [  
2    { name: 'Cranberries', preis: 0.40, amount: 65, pic:'' },  
3    { name: 'Ananasstücke', preis: 0.60, amount: 50, pic:'' },  
4    { name: 'Apfelringe', preis: 0.70, amount: 35, pic:'' },  
5    { name: 'Datteln', preis: 0.42, amount: 65, pic:'' },  
6  ]
```

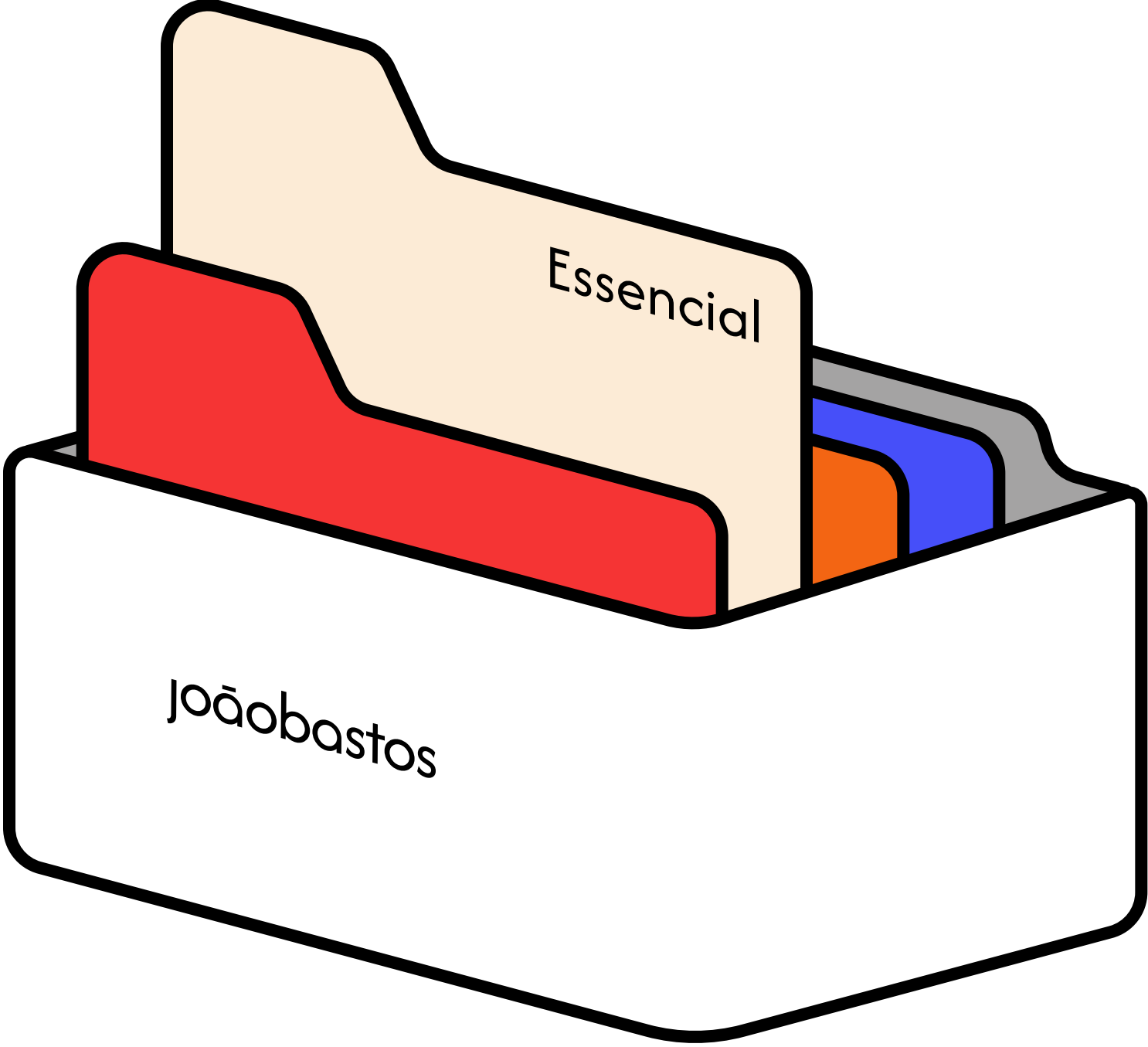
Und die Nutzung ist ebenso einfach.

```
1  arrayFruechte.forEach(product => {  
2    productListBasis.innerHTML += `  
3      <div class="product">  
4        <p class="product-name">${product.name}</p>  
5        <img src = "${product.pic}">  
6        <p class='menge'>${product.amount}g</p>  
7        <p class='preis'>${product.preis.toFixed(2)}€</p>  
8      </div>  
9    `;  
10  });
```







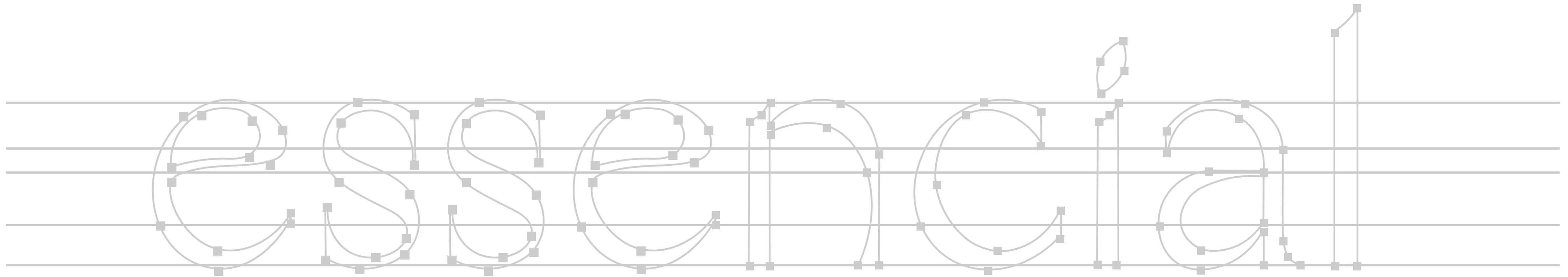




Essencial ist ein Massage-Salon, der Ruhe und Wohlbefinden in den Mittelpunkt stellt. Die visuelle Identität und die Website wurden von mir entwickelt.

Auf der Website werden die Professionalität und Philosophie der Therapeutin hervorgehoben sowie die verschiedenen Angebote übersichtlich präsentiert.





Design

Ruhig. Stark. Harmonisch

Für das Design habe ich eine Farbwelt entwickelt, die Ruhe und Stärke ausstrahlt.

Die Kundin wünschte sich einen minimalen Bezug zur brasilianischen Flagge und zur Herkunft der Massage. Durch die reduzierte und ausgewogene Farbpalette entsteht ein entspannter, chilliger Ton, der perfekt zum Salon passt.

Neulis Neue

Aa 123 # \$ *

Sand

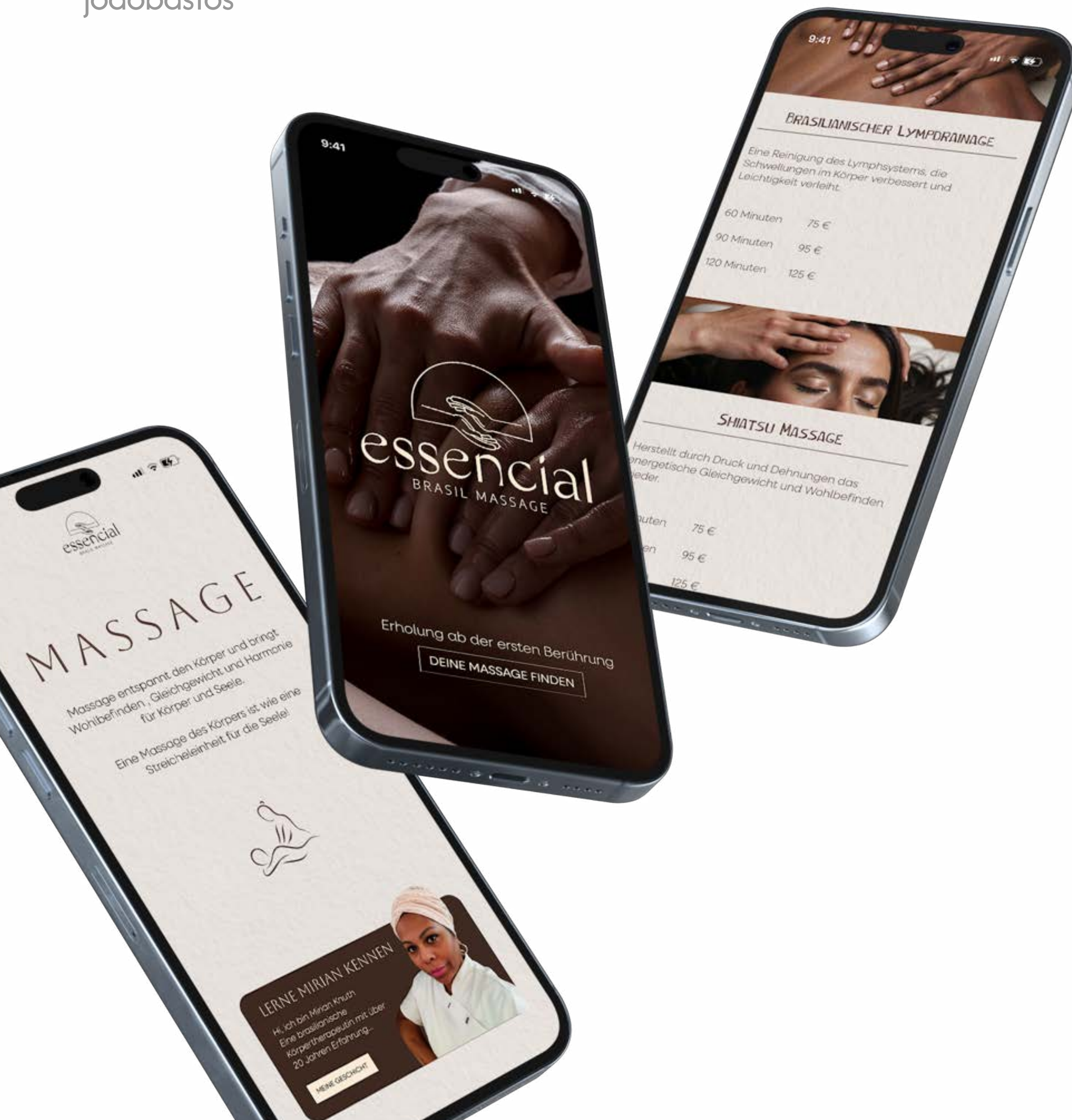
C: 49 R: 252 #FCEBD6
M: 61 G: 235
Y: 60 B: 214
K: 66

Nature

C: 49 R: 45 #2D3A18
M: 61 G: 58
Y: 60 B: 24
K: 66

Erde

C: 49 R: 67 #433023
M: 61 G: 48
Y: 60 B: 35
K: 66



Fronted

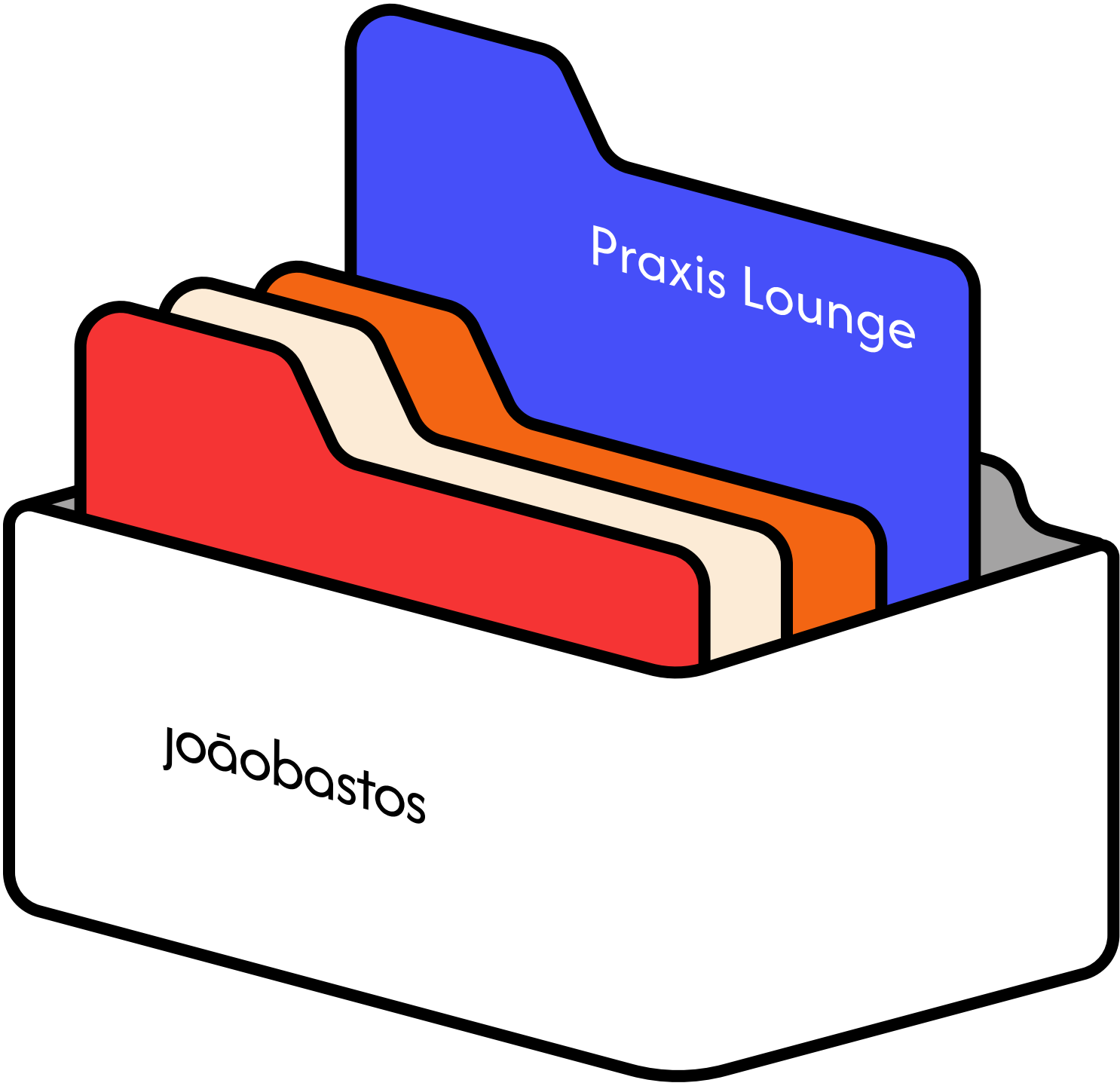
Ein personalisiertes Framework

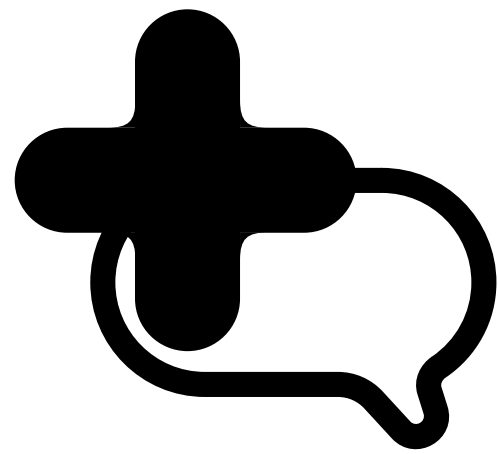
Mein Ziel beim Programmieren der Website von **Essencial** war es, den Code so klar und selbsterklärend wie möglich zu gestalten. Da die Kundin die Seite aktualisieren oder erweitern möchte, habe ich darauf geachtet, eine Struktur zu schaffen, die leicht verständlich und intuitiv ist. Man kann also sagen, dass ich im Grunde ein personalisiertes kleines Framework für Essencial entwickelt habe.

Für das Hosting habe ich mich für **Hostinger** entschieden, weil es die Pflege praktisch und leicht macht. Gleichzeitig ist Hostinger auch im Bereich Sicherheit gut aufgestellt.





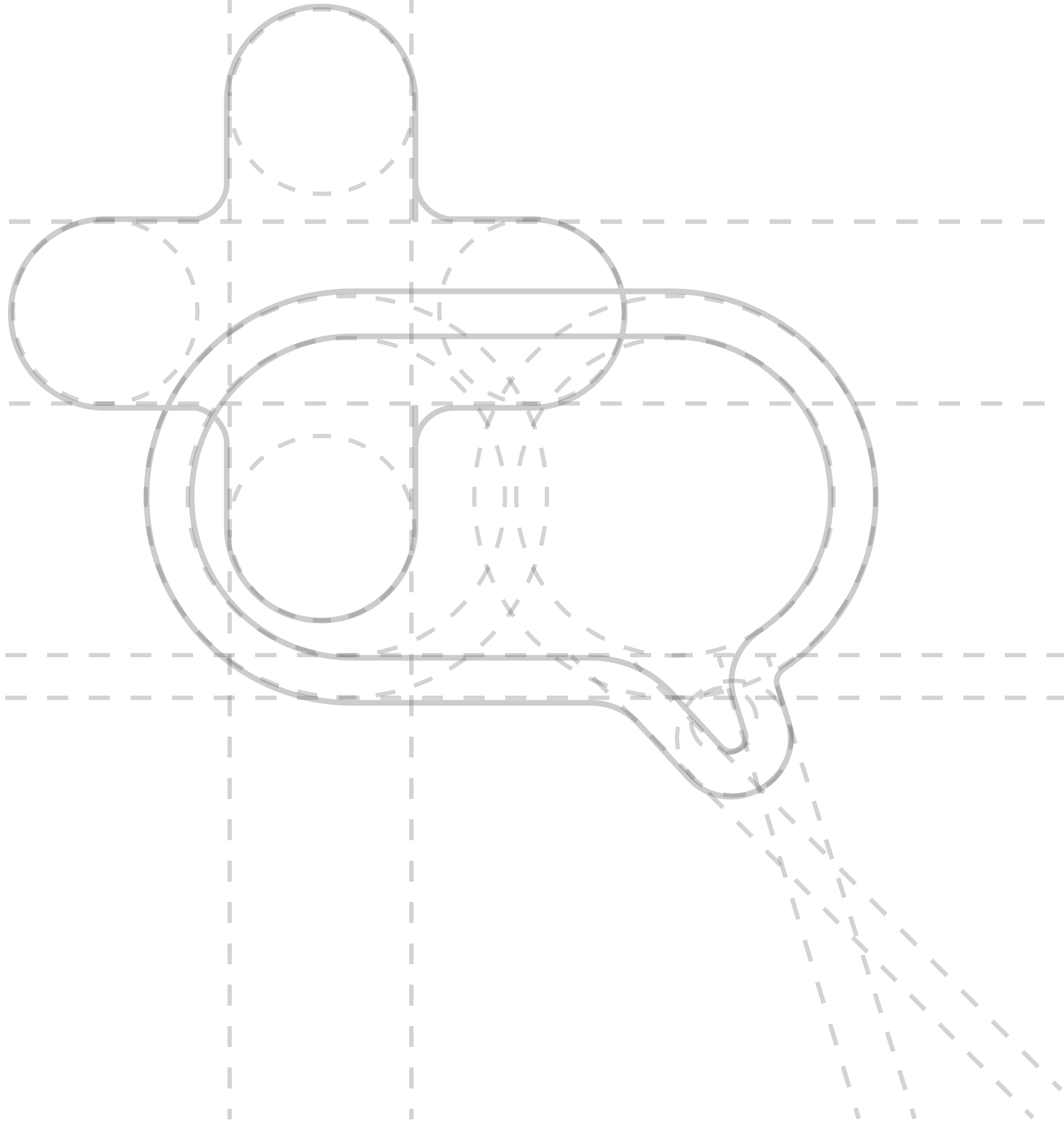




Praxis Lounge ist eine mobile-first Chat-App für den Krankenhausalltag. Sie ermöglicht eine schnelle und klare Kommunikation zwischen Pflegekräften, selbst wenn sie auf verschiedene Stationen verteilt sind.

Die größte Herausforderung liegt im Backend, das eine sichere, stabile und performante Echtzeit-Kommunikation ermöglichen muss.





Design

Leichtigkeit und Vertrauen

Für das Design von Praxis Lounge habe ich eine ruhige und harmonische Farbwelt entwickelt, die auf verschiedenen Blau- und Lilatönen basiert. Diese Farben wurden bewusst gewählt, um den Charakter eines Gesundheitsumfelds widerzuspiegeln.

Das Logo vereint ein Chat-Symbol mit dem Kreuz, dem klassischen Zeichen des Gesundheitswesens. Diese Kombination steht für Kommunikation und Pflege

Candal
Poppins

Aa 123 ¢\$*

Aa 123 # \$ *

Dark

Secondary

Primary

Accent

Light

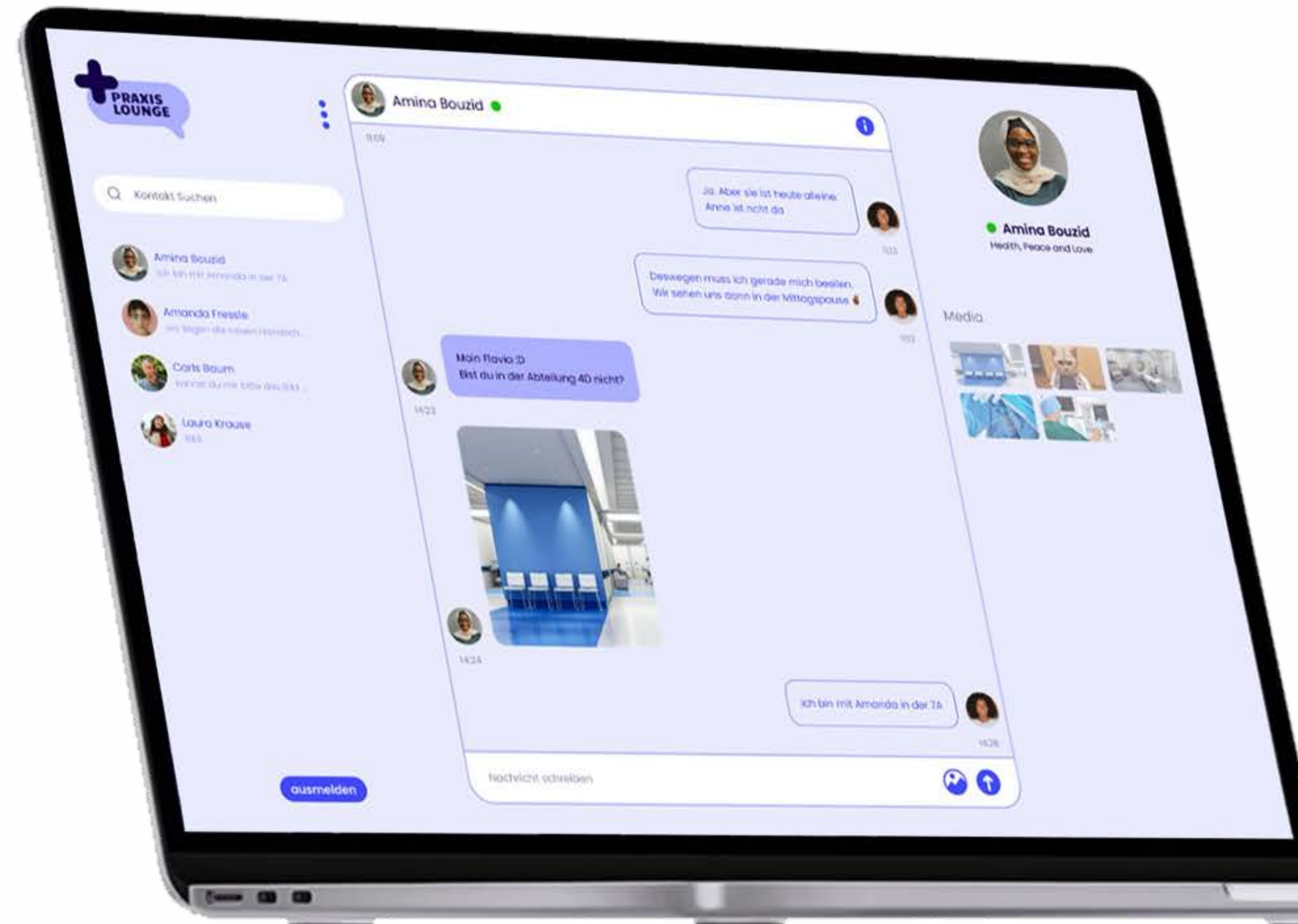
Backend

Für das Backend habe ich Firebase Firestore und SQL verwendet.

Ich habe meine React-App mit Firestore verbunden, indem ich `db` aus `../config/firebase` importiert habe.

Dadurch werden im Firestore die Datenbank-Table erstellt und verwaltet.

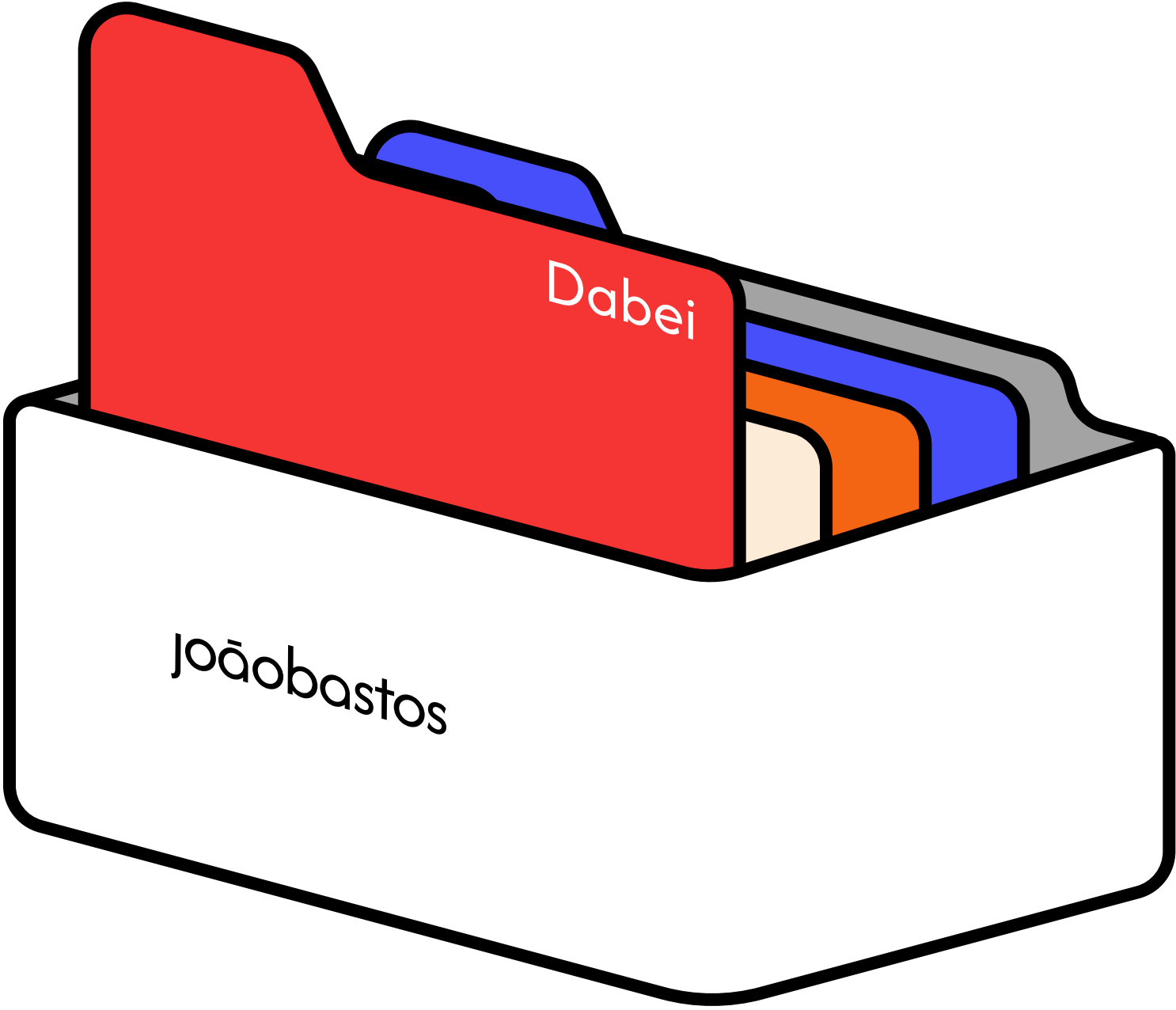
```
1  import { doc, getDoc } from "firebase/firestore";
2  import { db } from "../config/firebase";
3
4  const chatsTable = doc(db, 'chats', uid);
5  const chatsSnap = await getDoc(chatsTable);
6  setChatsData(chatsSnap.data());
7
8  const chatsRef = collection(db, "chats");
9
10 await updateDoc(doc(chatsRef, user.id), {
11   chatsData: arrayUnion({
12     messageId: newMessageRef.id,
13     lastMessage: "",
14     rId: userData.id,
15     updatedAt: Date.now(),
16     messageSeen: true,
17   }),
18 });
```



Durch den Einsatz von React-Bibliotheken in Kombination mit Firebase lässt sich die Entwicklung effizient umsetzen. Ein Beispiel dafür ist die Benutzerregistrierung, die mithilfe der signup-Funktion und Firestore-Collections implementiert wurde.

```
1  import { createUserWithEmailAndPassword } from "firebase/auth";
2  import { doc, setDoc } from "firebase/firestore";
3  import { auth, db } from "../config/firebase";
4
5  const signup = async (username, email, password) => {
6    const res = await createUserWithEmailAndPassword(auth, email,
7  password);
8    const user = res.user;
9
10   await setDoc(doc(db, "users", user.uid), {
11     id: user.uid,
12     username: username.toLowerCase(),
13     email,
14     name: "",
15     avatar: "",
16     bio: "Hallo, ich nutze jetzt PraxisLounge",
17     lastSeen: Date.now(),
18   });
```

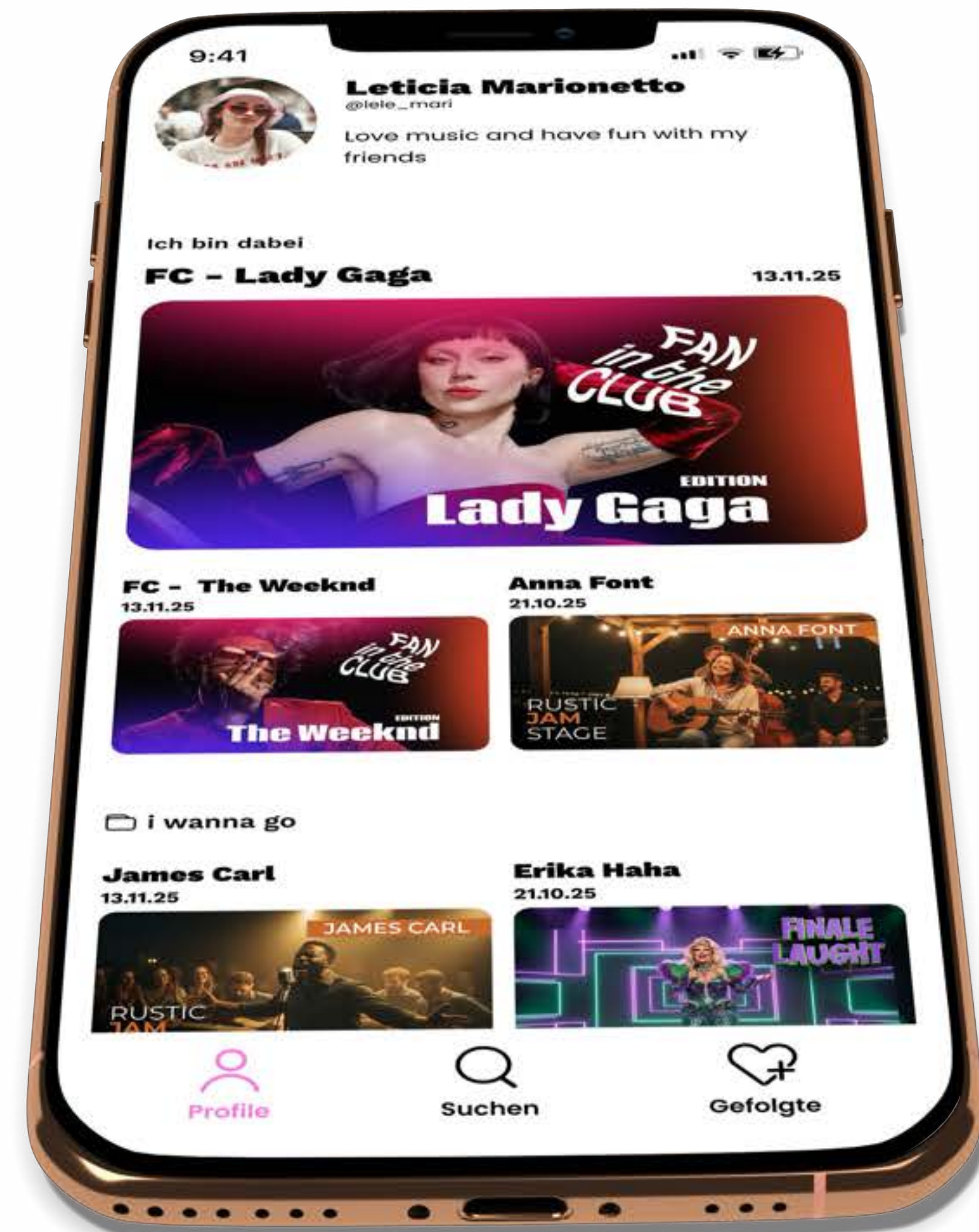


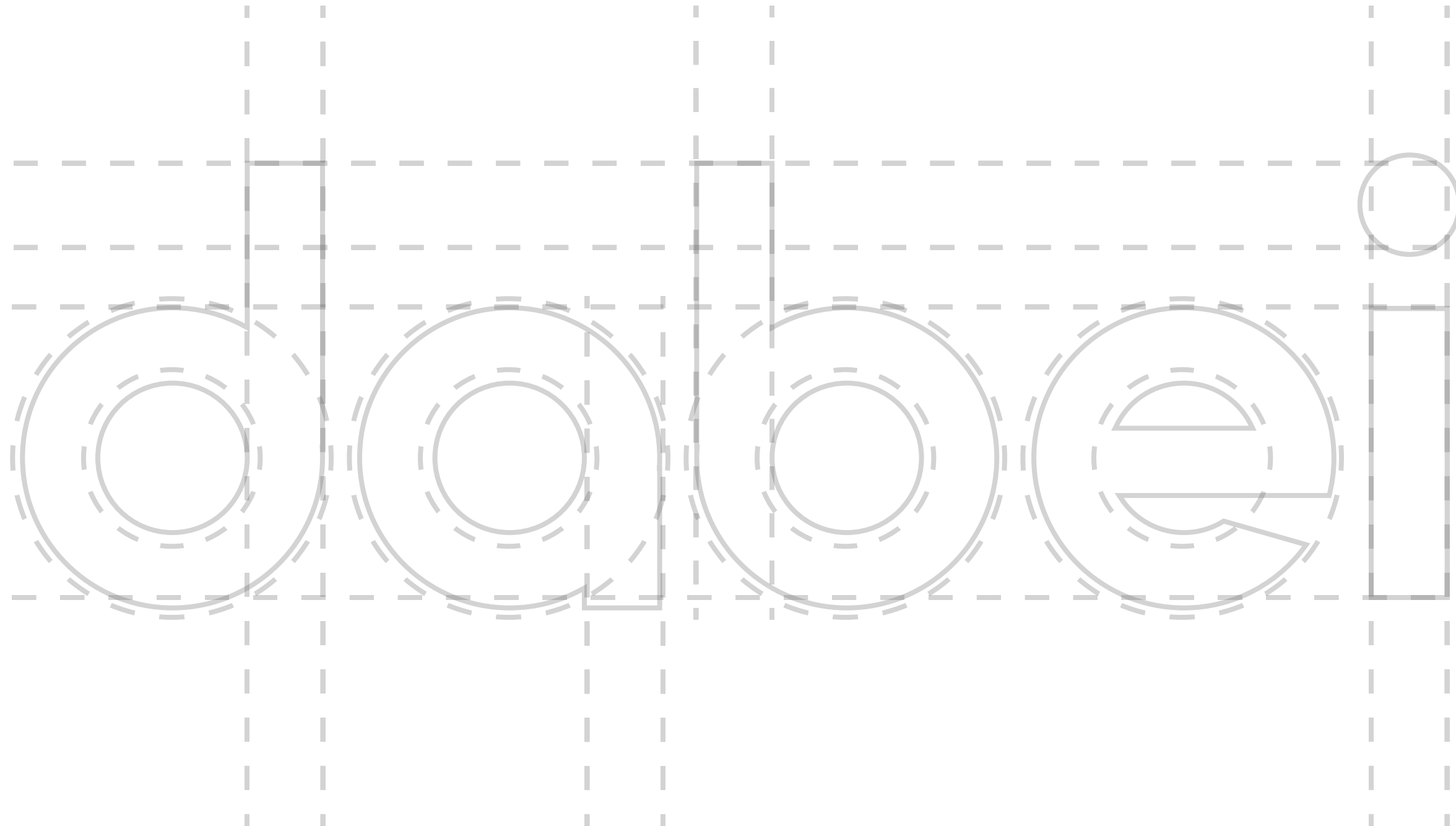



dabei

Dabei ist ein Social-Media-Netzwerk, das dich über alle Events in deiner Nähe auf dem Laufenden hält. Entdecke, was gerade um dich herum passiert, folge deinen Freunden oder den Hosts deiner Lieblingsveranstaltungen. Verpasse nie wieder ein spannendes Event.

Der MVP wird derzeit entwickelt. Vom Design bis zum Frontend und Backend.





Funktionalität

verpass nie wieder ein Event

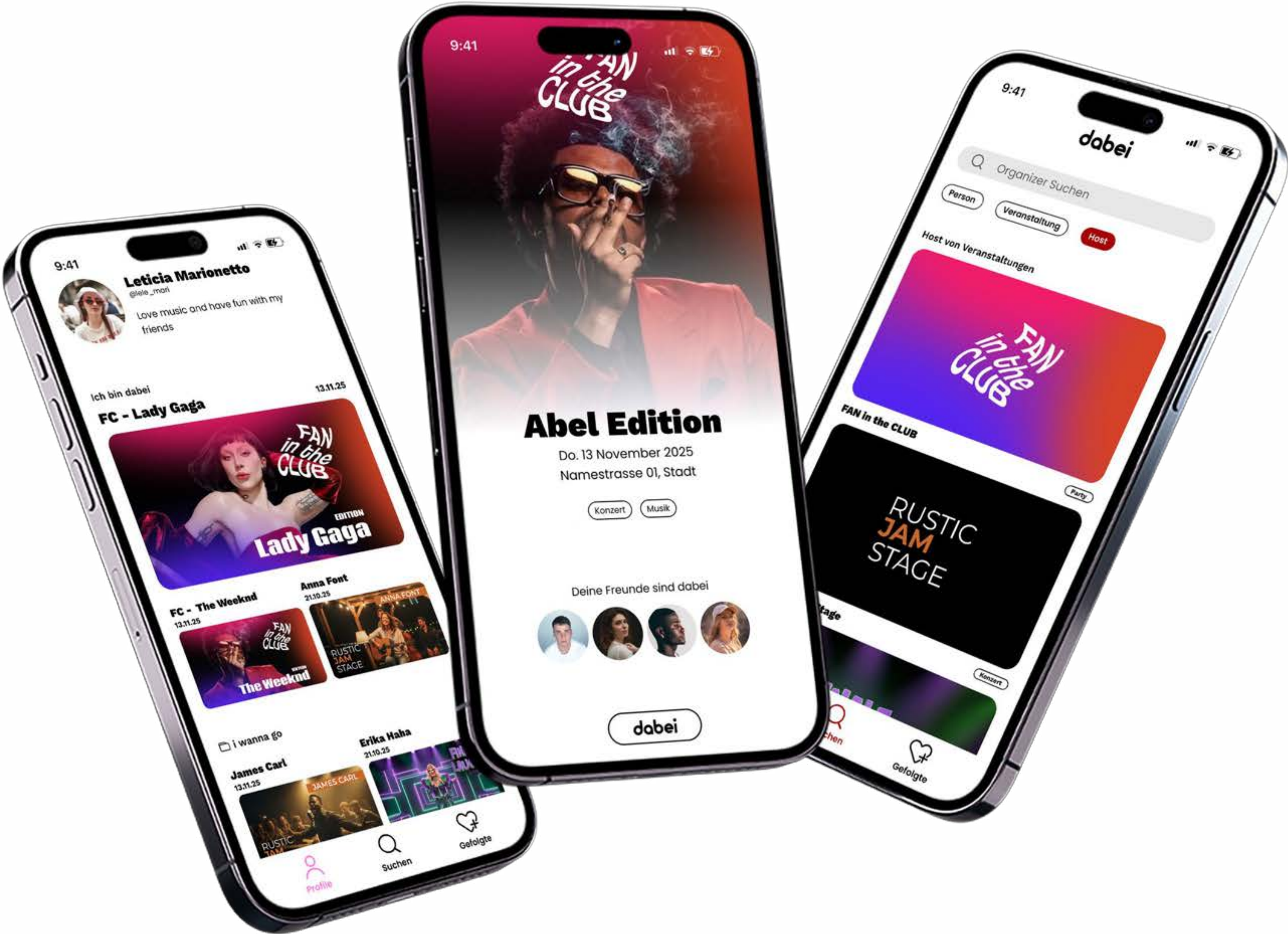
Neue Nutzer können sich ganz einfach registrieren, während bestehende Nutzer sich einfach einloggen.

Danach lässt sich das Profil individuell gestalten:

Man kann ein Profilbild vom Gerät hochladen, den Namen anpassen und eine kurze Bio hinzufügen.

Auf der Startseite kann man nach Kolleg:innen suchen und direkt einen Chat starten. Im Chat lassen sich Nachrichten und Fotos versenden.

*Alle Hosts und Veranstaltungen sind fiktiv und wurden von mir für dieses Projekt erstellt.



Backend

Für das Backend habe ich Python mit dem Framework Django und einer SQL-Datenbank verwendet.

Django ermöglicht durch seine Model-Struktur eine effiziente und saubere Erstellung sowie Verwaltung der Datenbank. Dadurch lassen sich Datenmodelle klar definieren, automatisch migrieren und konsistent im gesamten Projekt nutzen.

```
1  from django.db import models
2  from django.contrib.auth.models import User

3  class Event(models.Model):
4      host = models.ForeignKey(
5          User, on_delete=models.CASCADE, related_name='events'
6      )
7      titel = models.CharField(max_length=200)
8      image = models.ImageField(upload_to='event_images/',
9      blank=True, null=True)
10     adresse = models.CharField(max_length=255)
11     datum = models.DateTimeField()
12     beschreibung = models.TextField(blank=True)
13     event_tag = models.CharField(max_length=100, blank=True)
14     dabei_users = models.ManyToManyField(
15         User, related_name='dabei_events', blank=True
16     )
```


Durch die Verwendung von “Class” zur Definition der Datenbankmodelle von Django kann ich diese in jedem Python-App-Modul importieren und wiederverwenden.

Das ermöglicht eine dynamische Nutzung der Daten und gibt mir zugleich mehr Kontrolle und Übersicht darüber, was im Code und in der Datenbank passiert.

```
1  from django import forms
2  from .models import Event

3  class EventForm(forms.ModelForm):
4      class Meta:
5          model = Event
6          fields = [
7              'titel',
8              'image',
9              'adresse',
10             'datum',
11             'beschreibung',
12             'event_tag'
13         ]
14         widgets = {
15             'datum': forms.DateTimeInput(attrs={'type': 'datetime-local'}),
16         }
```



Danke für den Besuch

**möchtest du mit mir
arbeiten?**

email: joaobastos@outlook.de

[zu meinem LinkedIn](#)

[zu meinem Github](#)

[Schau dir den Portfolio in meiner Website](#)

